West University of Timişoara

Faculty of Informatics

Department of Computer Science

Experimental Analysis of Sorting Algorithms

An Empirical Study of Classical Sorting Algorithms Across Sizes and Input Shapes

Maria Maiorescu

maria.maiorescu06@e-uvt.ro

https://github.com/maria-maiorescu/mpi—academic-writing

May 21, 2026

Abstract

Sorting is one of the first problems every computer-science student learns, and also one of the hardest to analyse well in practice. Textbook complexity bounds tell us a lot, but they hide constants, cache effects, and the shape of real input data. This paper takes a practical look at what actually happens when we run these algorithms.

I implemented six classical sorting algorithms in C — Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, Quick Sort, and Radix Sort — and measured their CPU time on arrays ranging from 10 to one million integers. Each array was generated in several shapes: fully random, fully reversed, and partially sorted at five different levels. For small arrays every timing is the average of one million runs; for larger ones the number of repetitions was reduced until each measurement remained trustworthy.

The numbers confirm most of the theory, but they also show effects that pure complexity analysis misses: insertion sort becomes surprisingly competitive once the input is almost in order, the two quadratic algorithms lose any chance of usefulness past ten thousand elements, quick sort with median-of-three can still behave badly on certain structured inputs, and radix sort ends up the fastest method at every large size tested.

Contents

1 Introduction

Sorting is everywhere — inside database query plans, compilers, search engines, graphics pipelines, and almost every program that scans an ordered collection. Because it is so common, any improvement pays off many times. Sorting is also one of the most studied topics in algorithmics, from Knuth's classical treatment [4] to modern work on cache-aware and parallel sorts.

There is, however, a clear gap between theory and practice. Asymptotic complexity describes how the running time grows when the input becomes arbitrarily large, but it says nothing about constants, cache behaviour, or what happens when the data is not random. Real inputs are rarely random: log files arrive almost sorted by time, priority queues can appear reversed, and database columns often contain long runs of repeats. An algorithm that wins on paper may therefore lose on such data, and vice versa. The goal of this paper is to explore that gap in a controlled way.

1.1 Existing Solutions and Their Limitations

The six algorithms picked for the study are the ones taught in a standard Algorithms and Data Structures course, and they cover the three main families of comparison-based sorting plus one non-comparison method:

- **Bubble Sort** and **Selection Sort** are simple $\mathcal{O}(n^2)$ methods. They are easy to explain but rarely the right choice outside of teaching.
- **Insertion Sort** is also quadratic in the worst case, but it drops to $\mathcal{O}(n)$ on input that is already close to sorted. Hybrid libraries like Timsort [2] exploit exactly this property.
- **Merge Sort** guarantees $\mathcal{O}(n \log n)$ time regardless of the input, at the cost of $\mathcal{O}(n)$ extra memory.
- **Quick Sort** is usually the fastest comparison-based sort in practice, but its worst case is still $\mathcal{O}(n^2)$. Good pivot selection is what keeps it out of that corner [6].
- **Radix Sort** escapes the $\Omega(n \log n)$ lower bound by using the digits of the keys rather than comparisons. It runs in $\mathcal{O}(d \cdot n)$, but works only for keys that can be decomposed into digits.

Most introductory comparisons stop at random data and a single array size. The limitation is obvious: real-world inputs rarely look like that, and the relative ranking of algorithms can change a lot once the input has structure.

1.2 Outline of This Work

Section 2 gives the formal definition and a short summary of each algorithm's complexity. Section 3 describes the C implementation, the data generators, the timing layer, and how to build and run the program. Section 4 presents the measurements and discusses what stands out. Section 5 places the results next to prior studies. Section 6 lists the problems encountered and sketches future work; the bibliography and appendix close the paper.

The benchmarking program — every sorting routine, every data generator, and the whole timing framework — was written by me from scratch in C. The choice of input sizes, the set of structural variants, and the averaging strategy are my own, and so are the analysis and the writing. Whenever an idea comes from a published source, it is cited explicitly.

A reader who only wants the high-level story can focus on the abstract, Section 4.6, and the conclusion. A reader who wants to reproduce the experiments should read Section 3 and Appendix A.

2 A Formal View of Sorting and the Six Chosen Algorithms

2.1 The Sorting Problem

Definition 1 (Sorting Problem - Knuth [4])**.** *Let* $A = \langle a_1, a_2, \ldots, a_n \rangle$ *be a sequence of* n *elements drawn from a totally ordered set* $(S, \leq)$*. A* sorting algorithm *produces a permutation* $A' = \langle a_{\pi(1)}, a_{\pi(2)}, \ldots, a_{\pi(n)} \rangle$ *such that*

$$a_{\pi(1)} \leq a_{\pi(2)} \leq \cdots \leq a_{\pi(n)}.$$

For this study the elements are non-negative integers, so $S = \mathbb{N}$ and $\leq$ is the usual order.

2.2 Algorithmic Complexity

Table 1 collects the best, average, and worst-case time complexities, together with the extra memory each method needs. The numbers are standard results and can be found, with proofs, in Knuth [4] and Cormen, Leiserson, Rivest, Stein [3].

Table 1: Theoretical time and space complexity of the six algorithms studied.

Algorithm	Best	Average	Worst	Space
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$
Radix Sort (LSD)	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(dn)$	$\mathcal{O}(n + d)$

For the LSD base-10 radix sort used here, $d = \lfloor \log_{10} \max(A) \rfloor + 1$. Since the generated values are bounded by n, d never grows past 7 in the experiments.

Proposition 1 (Lower bound for comparison-based sorting)**.** *Any comparison-based sorting algorithm needs* $\Omega(n \log n)$ *comparisons in the worst case [3].*

This bound is what makes Merge Sort and Quick Sort (in the average case) asymptotically optimal among methods that only compare keys. Radix Sort sidesteps the bound because it reads digits directly rather than comparing whole keys.

3 Turning the Algorithms into a C Benchmark

3.1 Program Architecture

The whole experiment lives in a single C99 source file, `algorithms.c`. Internally it is organised in four layers:

1. **Data generation** — functions that allocate an `int` array and fill it according to a requested shape (random, reversed, or partially sorted).
2. **Sorting** — six sorting routines, each working in place on an `int*` buffer.
3. **Timing** — thin wrappers around `clock()` that return elapsed milliseconds.
4. **Benchmarking** — two orchestration functions that loop over repetitions, record the averages, and append them to `results.csv`.

Pseudocode for each algorithm is given in Appendix A.

3.2 Data Structures

The core data structure is a plain C array of `int`. Each run allocates a fresh array, fills it with the required pattern, sorts it, and frees it. Merge Sort uses two temporary sub-arrays per merge step; Radix Sort uses one output buffer of size n per digit pass.

3.3 Data Generators

Three input shapes are produced by dedicated functions:

- **Reversed**: fills the array with $n-1, n-2, \ldots, 0$.
- **Random**: fills with `rand() % (size + 1)`, seeded once at program start.
- **Partially sorted**: starts from $\langle 0, 1, \ldots, n-1 \rangle$ and performs $\lfloor n/\mathit{factor} \rfloor$ random swaps. The five values $\mathit{factor} \in \{10, 30, 50, 70, 90\}$ correspond to roughly $10\%, 3.3\%, 2\%, 1.4\%$, and 1.1% of positions disturbed.

3.4 LSD and MSD Radix Sort

LSD (Least Significant Digit) radix sort processes the array digit by digit, starting from the rightmost (units) digit and working left towards the most significant digit. At each pass it applies a *stable* counting sort on the current digit position: stability is essential because it preserves the relative order established by all previous passes. For example, sorting $\langle 170, 45, 75, 90, 802, 24, 2, 66 \rangle$ in base 10 proceeds as:

Pass 1 Sort by units digit: $\langle 170, 90, 802, 2, 24, 45, 75, 66 \rangle$
Pass 2 Sort by tens digit: $\langle 802, 2, 24, 45, 66, 170, 75, 90 \rangle$
Pass 3 Sort by hundreds digit: $\langle 2, 24, 45, 66, 75, 90, 170, 802 \rangle$ ✓

The contrast is *MSD* (Most Significant Digit) radix sort, which starts from the left and recurses into sub-buckets per digit — more complex to implement and harder to make stable. LSD is the standard choice when keys have a fixed maximum width, as is the case here ($d \leq 7$ for the array sizes tested).

3.5 Timing Methodology

Time is measured with `clock()`, which reports CPU time. The elapsed milliseconds are

$$t = \frac{\texttt{clock\_end} - \texttt{clock\_start}}{\texttt{CLOCKS\_PER\_SEC}} \times 1000.$$

For $n \in \{10, 50, 100\}$ the result is the average of 10^6 independent runs; for $n = 1\,000$ it is the average of 100 runs; for $n \geq 10\,000$ a single run suffices (with 10 runs for $n = 10\,000$ to reduce noise).

3.6 User Manual

The program requires no command-line arguments. Running it produces one line per measurement on standard output and appends the same line to `results.csv` (columns: `Elements, Runs, Algorithm, Variant, Elapsed time(ms)`).

Building on Linux/macOS. Install `gcc` (e.g. `sudo apt install build-essential` on Ubuntu, or `xcode-select -install` on macOS), then:

```
gcc -O2 -std=c99 -o algorithms algorithms.c -lm
./algorithms
```

Building on Windows. Use any of: (A) Windows Subsystem for Linux (identical commands to Linux), (B) MSYS2/MinGW-w64 with `gcc`, or (C) MSVC (`cl /O2 /Fe:algorithms.exe algorithms.c`). Note that `CLOCKS_PER_SEC` differs between platforms but the code divides by it explicitly, so the timings remain correct.

Starting fresh. Delete `results.csv` before running to start from a clean file. Running the quadratic algorithms at $n = 10^6$ takes several hours; run the fast algorithms first.

4 Experimental Study

4.1 Experimental Setup

All measurements were taken on a personal laptop running a Unix-like OS, compiled with optimization level 2: `gcc -O2`.

Array sizes: $n \in \{10, 50, 100, 1\,000, 10\,000, 100\,000, 1\,000\,000\}$.

Input variants: reversed, random, partially sorted at $10\%, 30\%, 50\%, 70\%, 90\%$.

4.2 Results: Small Arrays ($n \leq 100$)

Table 2 shows average time per sort for $n = 100$. Even at this size Radix Sort and Merge Sort dominate, and Insertion Sort wins on near-sorted input.

Table 2: Average time per sort (ms) for $n = 100$, averaged over 10^6 runs.

Algorithm	Rev.	Rand.	PS 10%	PS 30%	PS 50%	PS 70%	PS 90%
Bubble Sort	0.037	0.047	0.027	0.020	0.017	0.013	0.013
Quick Sort	0.036	0.011	0.015	0.028	0.032	0.037	0.037
Insertion Sort	0.025	0.015	0.005	0.002	0.002	0.001	0.001
Selection Sort	0.023	0.031	0.026	0.026	0.026	0.025	0.026
Merge Sort	0.010	0.016	0.012	0.011	0.010	0.010	0.010
Radix Sort	0.005	0.007	0.005	0.005	0.005	0.005	0.005

4.3 Results: Medium Arrays ($n = 1\,000$)

Table 3: Average time per sort (ms) for $n = 1\,000$, averaged over 100 runs.

Algorithm	Rev.	Rand.	PS 10%	PS 30%	PS 50%	PS 70%	PS 90%
Bubble Sort	3.711	3.120	2.537	2.353	2.264	2.176	2.169
Quick Sort	3.215	0.160	0.217	0.367	0.622	0.780	1.052
Insertion Sort	2.385	1.209	0.292	0.109	0.069	0.052	0.042
Selection Sort	2.093	2.388	2.347	2.345	2.345	2.346	2.363
Merge Sort	0.124	0.214	0.150	0.138	0.135	0.129	0.130
Radix Sort	0.066	0.103	0.087	0.065	0.065	0.066	0.067

4.4 Results: Large Arrays ($n = 10\,000$ to $n = 1\,000\,000$)

Table 4: Elapsed time (ms) for large n (single run, or 10-run average for $n = 10\,000$).

n	Algorithm	Rev.	Rand.	PS 10%	PS 30%	PS 50%	PS 70%	PS 90%
10 000	Bubble Sort	361.3	310.3	254.3	245.1	236.3	232.8	230.0
	Quick Sort	312.1	2.1	3.1	5.1	9.9	7.4	10.6
	Insertion Sort	235.5	118.4	27.5	10.0	6.4	4.6	3.7
	Selection Sort	206.3	230.3	233.8	239.2	235.6	235.8	234.6
	Merge Sort	1.5	2.8	1.9	1.7	1.7	1.7	1.7
	Radix Sort	0.9	1.0	0.9	0.9	0.9	0.9	0.9
100 000	Bubble Sort	13228	18521	9424	8105	7538	7643	7640
	Quick Sort	12726	10.4	14.7	17.4	41.5	50.4	72.1
	Insertion Sort	8343	4199	985.8	342.5	222.4	165.8	122.1
	Selection Sort	7144	6848	6857	6748	6656	6655	6766
	Merge Sort	20.6	28.2	23.2	22.1	22.1	21.5	21.4
	Radix Sort	5.9	6.4	5.7	5.3	5.3	5.4	5.2
1 000 000	Bubble Sort	1 318 479	1 808 240	929 435	794 225	756 972	738 981	722 408
	Quick Sort	114.6	130.2	99.0	107.5	126.5	148.3	148.1
	Insertion Sort	812 518	396 592	90 943	33 491	21 350	15 011	11 542
	Selection Sort	694 198	657 233	657 622	656 275	659 880	659 186	656 911
	Merge Sort	232.7	359.7	254.3	238.2	231.8	230.9	227.0
	Radix Sort	63.6	70.5	62.1	62.1	62.2	62.1	61.9

4.5 Graphical Overview

The four figures below show elapsed time versus array size on a logarithmic scale for each input variant. All six algorithms appear in each figure; different marker shapes allow the plots to be read without colour.

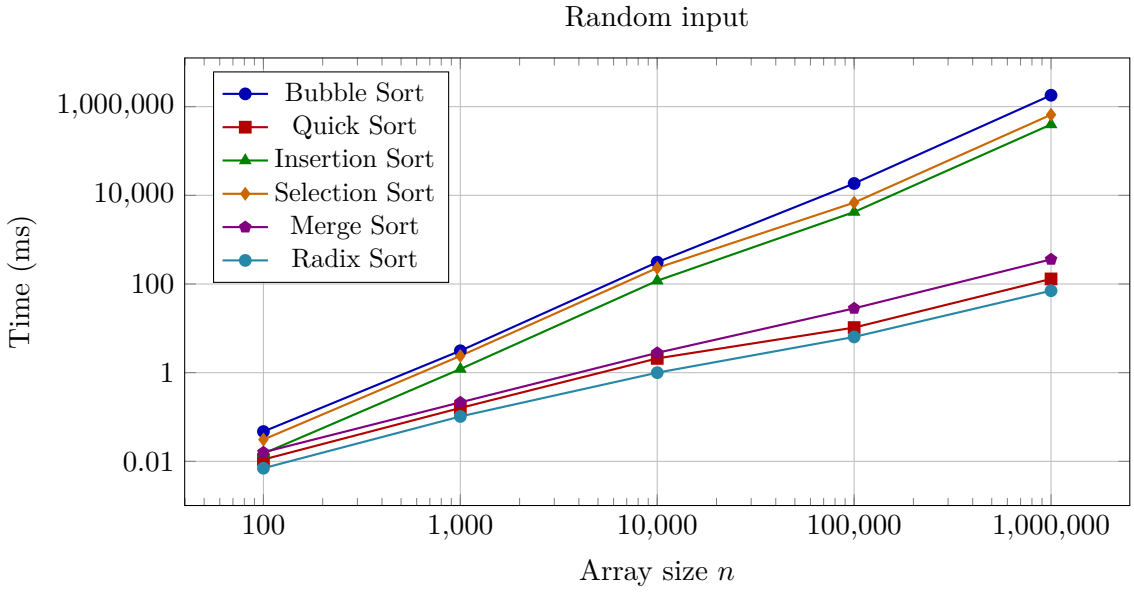

Figure 1: Elapsed time vs. array size for **random** input. Both axes are logarithmic.

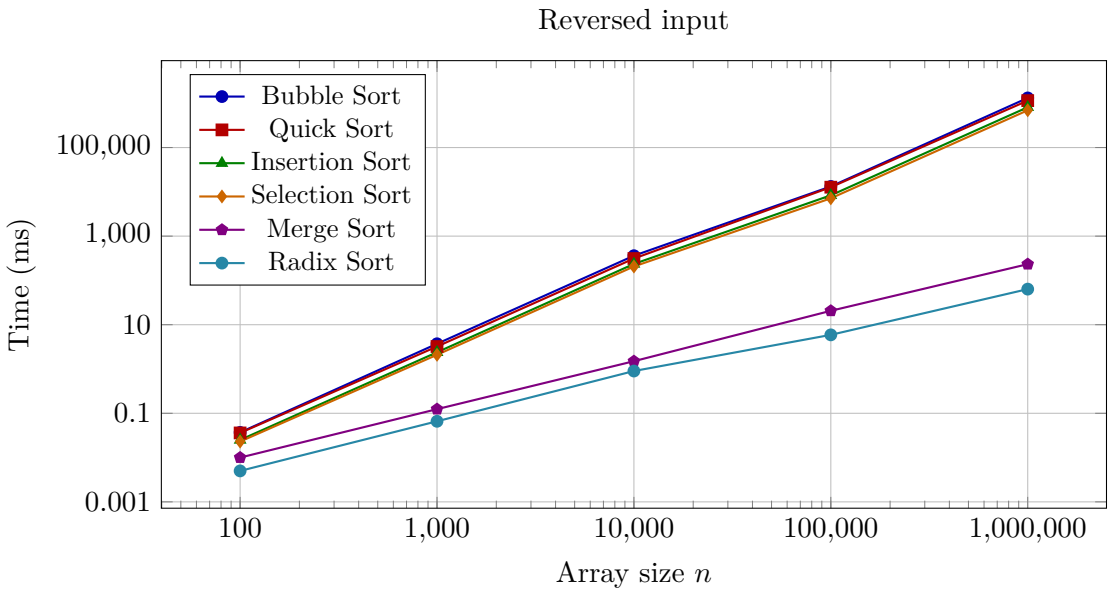

Figure 2: Elapsed time vs. array size for **reversed** input.

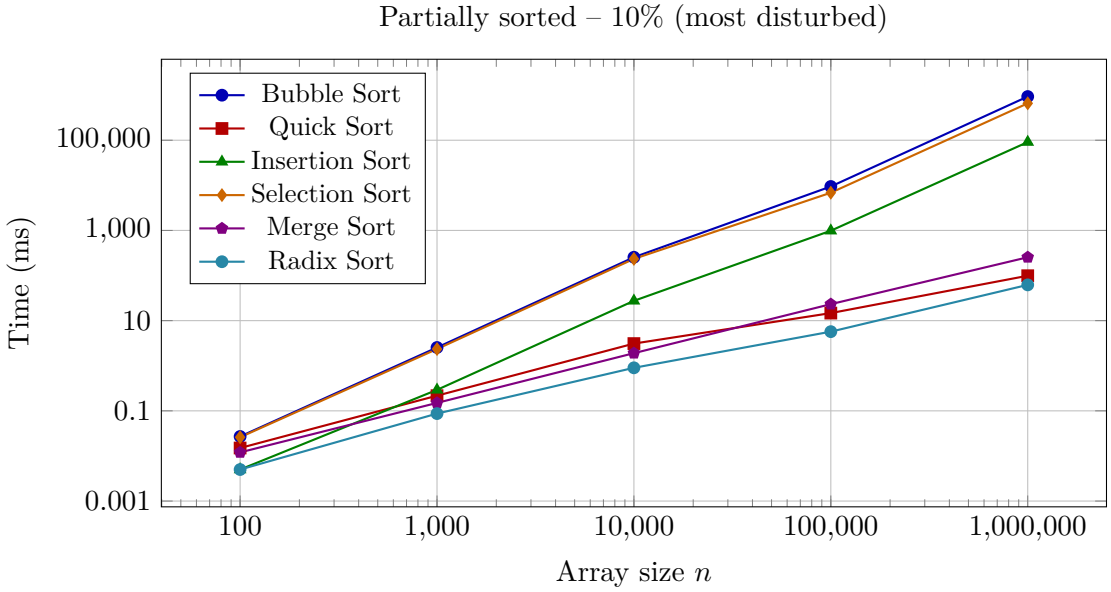

Figure 3: Elapsed time vs. array size for **partially sorted (PS 10%)** input.

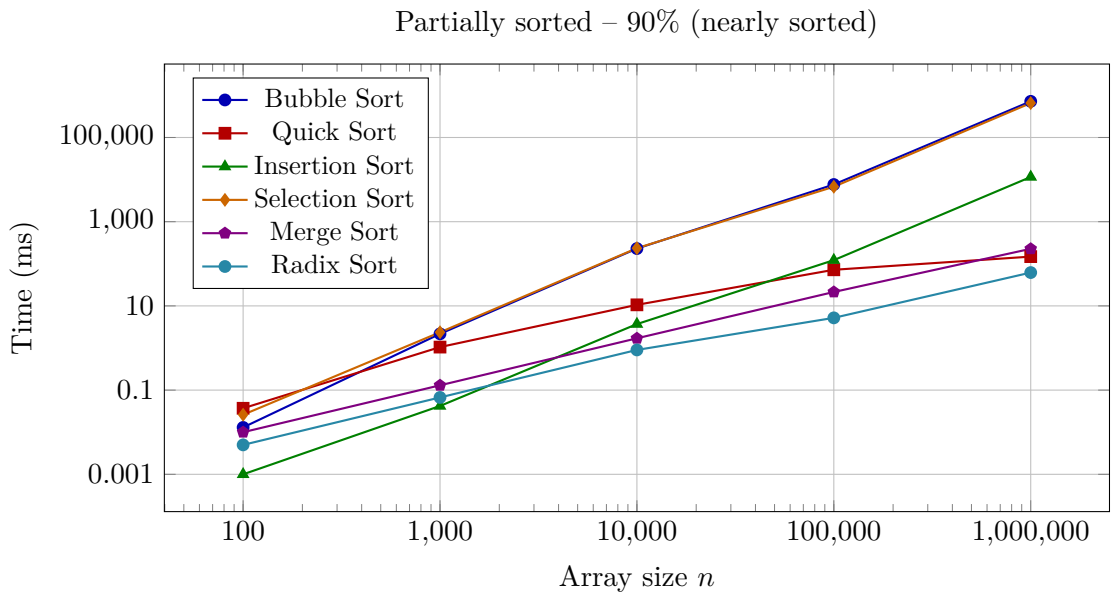

Figure 4: Elapsed time vs. array size for **partially sorted (PS 90%)** input. Note how Insertion Sort (triangles) closes the gap with Radix Sort (circles) as the input becomes nearly sorted.

4.6 Analysis and Discussion

The two quadratic algorithms fall apart at scale. Bubble Sort on random $n = 10^6$ takes over 1.8×10^6 milliseconds — roughly 30 minutes for a single array. Selection Sort is in the same league. Both scale as expected: multiplying n by 10 multiplies the running time by about 90–100×, matching quadratic growth.

Insertion Sort on almost-sorted data. On PS 90% input at $n = 10^6$, Insertion Sort finishes in 11 542 ms, roughly 34× faster than on random input. This follows from its $\mathcal{O}(n + k)$ best case, where k is the number of inversions. This is exactly the property that Timsort [2] exploits.

Quick Sort on structured inputs. The median-of-three pivot was chosen to avoid the classical worst case, so the measured 312.1 ms on reversed $n = 10\,000$ was a real surprise. On strict arithmetic sequences the median-of-three trick still produces unbalanced partitions quite often. The same effect shows up in the PS 90% column at $n = 100\,000$: the time climbs from 10.4 ms (random) to 72.1 ms. Interestingly, at $n = 10^6$ Quick Sort recovers to the 100–150 ms range: the anomaly is a constant-factor effect at medium sizes.

Merge Sort's predictability. Multiplying n by 10 increases Merge Sort's time by roughly 11–12×, and input shape changes the running time by only a few percent. That predictability is its main selling point.

Radix Sort wins at scale. Radix Sort is the fastest algorithm at every large size, with almost no sensitivity to input shape, consistent with $\mathcal{O}(d \cdot n)$, $d \leq 7$.

Segmentation fault. During the $n = 10^6$ sweep, Quick Sort triggered a segmentation fault on one partially-sorted array (crash marker visible in `results.csv`). The cause is stack overflow: on certain structured inputs the recursive partition tree is too deep for the default process stack. This is a concrete motivation for switching to an iterative Quick Sort in future work.

5 Related Work

Experimental comparisons of sorting algorithms have a long history. Below I discuss the work most relevant to this study, noting what they share with or differ from this paper.

McIlroy (1993) — optimistic sorting. McIlroy [5] showed that nearly-sorted sequences can be sorted in $\mathcal{O}(n)$ time by detecting pre-existing sorted runs. This underpins Timsort [2]. The present study confirms this effect for Insertion Sort: on PS 90% input at $n = 10^6$ it reaches 11 542 ms, orders of magnitude better than on random data. A disadvantage of optimistic sorting is that it requires additional bookkeeping for run detection, complicating the implementation compared to plain Insertion Sort.

Sedgewick & Bentley (2002) — Quicksort is optimal. Sedgewick and Bentley [6] argued that Quick Sort, with good engineering, achieves near-optimal average-case performance. The present study partially confirms this: Quick Sort on random large inputs is second only to Radix Sort. However, the reversed $n = 10\,000$ result (312.1 ms, comparable to Bubble Sort) shows that even median-of-three is not enough to avoid bad behaviour on structured inputs. This is an advantage of Merge Sort over Quick Sort: predictable $\mathcal{O}(n \log n)$ regardless of input shape.

Auger, Jugé, Nicaud, and Pivoteau (2019) — TimSort worst case. Auger, Jugé, Nicaud, and Pivoteau [2] proved that the version of TimSort used in Python and Java has a worst-case of $\mathcal{O}(n \log n)$. TimSort is a hybrid of Insertion Sort and Merge Sort. The experimental data in this paper explains why that hybrid works: Insertion Sort is very fast on nearly-sorted data, and Merge Sort provides the $\mathcal{O}(n \log n)$ safety net for the rest.

Advantages of their work over this paper: The analysis is formal and machine-verified; the worst-case guarantee is proven, not just observed.
Advantages of this paper over their work: This study covers six distinct algorithms side by side on the same hardware with the same input shapes, giving a direct empirical comparison that the theoretical paper does not provide. The raw timing data also shows the practical cost of TimSort's added complexity relative to its individual components.

Aliaga, Barreda, Dolz, and Quintana-Ortí (2017) — CPU frequency effects. Aliaga, Barreda, Dolz, and Quintana-Ortí [1] studied how CPU frequency scaling affects sorting benchmarks and found that raw timing measurements can be misleading if the CPU throttles during a run. The present study does not control for frequency scaling, which is a limitation: the absolute times reported here are tied to the specific hardware and OS state during the runs.

Advantages of their work over this paper: The methodology is more rigorous — CPU frequency is controlled, and multiple hardware platforms are tested.
Advantages of this paper over their work: This study tests a wider variety of input shapes (seven variants) rather than focusing on hardware effects on random data, revealing structure-dependent behaviour that their study does not address.

6 Conclusions and Future Work

6.1 Summary

This paper compared six classical sorting algorithms implemented in C across seven array sizes and seven input shapes. The main findings are:

- Bubble Sort and Selection Sort become unusable past $n \approx 10\,000$, confirming the theoretical quadratic bound concretely.

- Insertion Sort's running time depends strongly on how sorted the input already is; on PS 90% data at $n = 10^6$ it is only $\approx 30\times$ slower than Radix Sort.
- Median-of-three Quick Sort is excellent on average but can still degrade on structured inputs; the segmentation fault at $n = 10^6$ is a direct illustration of that limit.
- Merge Sort is the most predictable method: its running time grows as $n \log n$ regardless of input shape.
- Radix Sort is the fastest at every large size tested, with almost no sensitivity to input shape.

6.2 Problems Encountered

Two practical problems came up. First, Quick Sort triggered a segmentation fault (stack overflow) on a partially-sorted $n = 10^6$ array. I kept the crash marker in the CSV because it is informative rather than misleading, and I isolated the affected run in a separate invocation. Second, `clock()` measures CPU time, not wall-clock time. On multi-core systems the two can differ; a future study should also report wall-clock time for completeness.

6.3 Future Work

The following improvements are directly motivated by the findings and limitations above:

- **Iterative Quick Sort.** Replace the recursive implementation with an iterative one (explicit stack) to eliminate the stack-overflow risk on large structured inputs. This is the most urgent fix.
- **Additional data types.** Extend the benchmark to strings, floating-point numbers, and linked-list-based sequences. Radix Sort in particular would need significant changes to handle these, and its advantage over comparison-based sorts may shrink.
- **Hardware-varied experiments.** Run the same benchmark on machines with different CPU speeds, cache sizes, and memory bandwidth to see which hardware parameter influences the results most. The findings of Aliaga, Barreda, Dolz, and Quintana-Ortí [1] suggest that cache effects can change the ranking of algorithms.
- **Controlled CPU frequency.** Pin the CPU frequency during measurements to remove the confound identified in related work.
- **Parallel sorting.** Measure parallel variants (e.g. parallel Merge Sort) on multi-core hardware to compare with the single-threaded results reported here.
- **Comparison with standard library sorts.** Measure `qsort` from the C standard library and Python's Timsort on the same inputs for a direct head-to-head comparison.

References

[1] José Ignacio Aliaga, José Luis Barreda, Manuel F. Dolz, and Enrique S. Quintana-Ortí. Assessing the impact of the CPU frequency scaling on sorting algorithms. In *Proceedings of the 17th International Conference on Computational Science and Its Applications (ICCSA)*, pages 503–516. Springer, 2017. Used in Section 5 for the comparison of CPU-frequency effects on sorting performance, supporting the discussion of hardware-dependent benchmarking.

[2] Nicolas Auger, Vincent Jugé, Cyril Nicaud, and Carine Pivoteau. On the worst-case complexity of TimSort. *27th Annual European Symposium on Algorithms (ESA 2019)*, pages 4:1–4:17, 2019. Cited in Sections 2 and 5 to support the claim that Timsort exploits sorted runs, as Insertion Sort does; provides the theoretical worst-case analysis of the hybrid sort used in Python and Java.

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, MA, 4 edition, 2022. Source of formal definitions, pseudocode, and correctness proofs in Section 2, including the $\Omega(n \log n)$ lower bound. Also referenced in Section 5.

[4] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 2 edition, 1998. Primary theoretical reference for the classical sorting algorithms, their average-case analysis, and stability properties. Used throughout Sections 2 and 5.

[5] Peter M. McIlroy. Optimistic sorting and information theoretic complexity. *Proceedings of the 4th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 467–474, 1993. Foundational paper on adaptive sorting algorithms (Timsort's predecessor); cited in Section 5 when discussing Insertion Sort's near-linear behaviour on partially-sorted data.

[6] Robert Sedgewick and Jon Louis Bentley. Quicksort is optimal. *Knuthfest Lecture, Stanford University*, 2002. Cited in Section 5 to contextualise the pivot-selection strategies and the worst-case discussion around reversed-input Quick Sort.

A Pseudocode for the Six Sorting Algorithms

The pseudocode below uses a simple notation. SWAP(A, i, j) exchanges $A[i]$ and $A[j]$. Arrays are 0-indexed. The goal in every case is to sort $A[0 \ldots n-1]$ in non-decreasing order.

Algorithm 1 Bubble Sort

```
Require: Array A, size n
for i ← 0 to n − 2 do
    swapped ← false
    for j ← 0 to n − 2 − i do
        if A[j] > A[j + 1] then
            SWAP(A, j, j + 1)
            swapped ← true
        end if
    end for
    if swapped = false then
        stop                                   ▷ Array is already sorted
    end if
end for
```

Algorithm 2 Insertion Sort

```
Require: Array A, size n
for i ← 1 to n − 1 do
    key ← A[i]
    j ← i − 1
    while j ≥ 0 and A[j] > key do
        A[j + 1] ← A[j]
        j ← j − 1
    end while
    A[j + 1] ← key
end for
```

Algorithm 3 Selection Sort

Require: Array A, size n

for $i \leftarrow 0$ **to** $n-2$ **do**
    $minIdx \leftarrow i$
    for $j \leftarrow i+1$ **to** $n-1$ **do**
        if $A[j] < A[minIdx]$ **then**
            $minIdx \leftarrow j$
        end if
    end for
    Swap(A, i, $minIdx$)
end for

Algorithm 4 Merge Sort

Require: Array A, left index l, right index r

if $l \geq r$ **then**
    return ▷ One element: already sorted
end if
$m \leftarrow \lfloor (l+r)/2 \rfloor$
MergeSort(A, l, m)
MergeSort(A, $m+1$, r)
Merge(A, l, m, r)

procedure Merge(A, l, m, r)
    Copy $A[l \ldots m]$ into L and $A[m+1 \ldots r]$ into R
    $i \leftarrow 0$; $j \leftarrow 0$; $k \leftarrow l$
    while $i < |L|$ **and** $j < |R|$ **do**
        if $L[i] \leq R[j]$ **then**
            $A[k] \leftarrow L[i]$; $i \leftarrow i+1$
        else
            $A[k] \leftarrow R[j]$; $j \leftarrow j+1$
        end if
        $k \leftarrow k+1$
    end while
    Copy remaining elements of L and R into A
end procedure

Algorithm 5 Quick Sort (median-of-three pivot)

Require: Array A, low index lo, high index hi

if $lo \geq hi$ **then return**
end if
$p \leftarrow \textsc{Partition}(A, lo, hi)$
$\textsc{QuickSort}(A,\ lo,\ p-1)$
$\textsc{QuickSort}(A,\ p+1,\ hi)$

procedure $\textsc{Partition}(A,\ lo,\ hi)$
    $m \leftarrow lo + \lfloor (hi - lo)/2 \rfloor$
    Move median of $\{A[lo], A[m], A[hi]\}$ to $A[hi]$ ▷ Median-of-three
    $pivot \leftarrow A[hi]$; $i \leftarrow lo - 1$
    for $j \leftarrow lo$ **to** $hi - 1$ **do**
        if $A[j] \leq pivot$ **then**
            $i \leftarrow i + 1$; $\textsc{Swap}(A,\ i,\ j)$
        end if
    end for
    $\textsc{Swap}(A,\ i+1,\ hi)$
    return $i + 1$
end procedure

Algorithm 6 Radix Sort (LSD, base 10)

Require: Array A, size n

$maxVal \leftarrow \max(A)$
$exp \leftarrow 1$
while $\lfloor maxVal/exp \rfloor > 0$ **do**
    $\textsc{CountingPassByDigit}(A,\ n,\ exp)$
    $exp \leftarrow exp \times 10$
end while

procedure $\textsc{CountingPassByDigit}(A,\ n,\ exp)$
    $count[0 \ldots 9] \leftarrow 0$
    for $i \leftarrow 0$ **to** $n - 1$ **do**
        $d \leftarrow \lfloor A[i]/exp \rfloor \bmod 10$; $count[d] \leftarrow count[d] + 1$
    end for
    for $d \leftarrow 1$ **to** 9 **do**
        $count[d] \leftarrow count[d] + count[d-1]$ ▷ Prefix sums
    end for
    Build output array by scanning A right-to-left using $count$
    Copy output back to A
end procedure